

Error Handling & Logging in Java

Introduction

Error handling and logging are critical aspects of robust java application development.

Try-Catch-Finally

Java uses try-catch-finally blocks to handle exceptions. The try block contains code that might throw an exception.

Ex:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
} finally {  
    System.out.println("Execution completed");  
}
```

Custom Exceptions

Java allows the creation of custom exceptions by extending the Exception class. This is useful for application-specific error handling.

Example:

```
class MyException extends Exception {  
    public MyException(String message) {  
        super(message);  
    }  
}
```

Logging Frameworks

Logging is essential for monitoring and debugging Java applications. Popular logging frameworks include:

- java.util.logging
- Log4j : full featured login framework; more secured
- SLF4J with Logback : Façade or abstraction layer; promote flexibility and decouples

Example using java.util.logging:

```
import java.util.logging.*;  
  
Logger logger = Logger.getLogger(MyClass.class.getName());  
  
logger.info("This is an info message");
```

Conclusion

Effective error handling and logging improve the maintainability and reliability of Java applications.